

CréditBot — Agent AI pour le Crédit Bancaire

1. Démarche suivie

Le projet vise à construire un agent intelligent spécialisé dans le crédit bancaire (crédit personnel, immobilier, taux, garanties, remboursement, éligibilité), en appliquant une approche **Agentic RAG** : le système ne se limite pas à une récupération documentaire suivie d'une génération, mais laisse le LLM raisonner, planifier et décider dynamiquement quels outils appeler.

Les étapes suivies :

- **Construction de la base documentaire** : rédaction de 4 documents PDF de référence sur le crédit personnel, le crédit immobilier, les garanties, et l'éligibilité/l'endettement, puis indexation via PyPDFLoader, découpage (RecursiveCharacterTextSplitter) et vectorisation (HuggingFaceEmbeddings, modèle sentence-transformers/all-MiniLM-L6-v2) dans un InMemoryVectorStore.
- **Développement des outils**: quatre tools (search_credit_kb, search_web, calculate_monthly_payment, check_eligibility), combinant récupération d'information et calcul métier.
- **Développement du graphe LangGraph** : création d'un état, d'un nœud agent (LLM + outils) et d'un nœud d'exécution des outils, reliés par des transitions conditionnelles pour permettre le raisonnement et l'utilisation des outils.
- **Ajout de la mémoire** : persistance de la conversation par thread_id via InMemorySaver, permettant à l'agent de conserver le contexte entre plusieurs échanges.
- Visualisation du graphe via draw_mermaid_png, et évaluation du système sur un jeu de 20 questions (10 simples, 10 complexes).

2. Fonctionnement du système

2.1 Architecture modulaire

Le code est organisé en modules indépendants : **state.py** (état partagé), **prompts.py** (system prompt métier), **rag.py** (pipeline RAG), **tools.py** (outils), **memory.py** (checkpointing), **graph.py** (construction et compilation du graphe), **app.py** (interface de chat) et **evaluation.py** (tests).

2.2 Boucle agentique (le graphe)

Le graphe démarre en START puis entre dans le nœud agent : le LLM lit le system prompt et l'historique, puis décide soit de répondre directement, soit d'appeler un ou plusieurs outils. Une arête conditionnelle (should_continue) redirige vers le nœud tool_node si un appel d'outil est détecté ; celui-ci exécute les outils demandés et renvoie leurs résultats sous forme de ToolMessage, avant de reboucler vers le nœud agent. Le cycle se répète jusqu'à ce que le LLM produise une réponse finale sans appel d'outil, ce qui déclenche la transition vers END.

2.3 Logique de décision RAG / Web

L'agent interroge d'abord la base documentaire interne (RAG) à l'aide de l'outil search_credit_kb. Si les informations sont insuffisantes ou si la question concerne des données récentes (taux directeur, réglementation, actualités), il utilise l'outil search_web. Les calculs, comme la mensualité ou le taux d'endettement, sont réalisés par des outils spécialisés afin de garantir des résultats fiables.

2.4 État et mémoire

l'état CreditAgentState conserve l'historique de la conversation (messages), le nombre d'appels au LLM, le nombre d'appels aux outils ainsi que les sources consultées. La mémoire InMemorySaver, associée à un thread_id, permet à l'agent de conserver le contexte et de poursuivre une conversation sans perdre les échanges précédents.

2.5 Visualisation

Le graphe compilé et généré automatiquement au lancement de app.py (fichier graph.png).

3. Résultats obtenus de l'évaluation

Le script evaluation.py exécute automatiquement 10 questions simples (définitions et notions de base couvertes par la base documentaire) ainsi que 10 questions complexes (calculs, comparaisons et questions nécessitant éventuellement plusieurs outils ou une recherche web). Pour chaque question, il mesure le temps de réponse, le nombre d'appels au LLM, le nombre d'appels aux outils ainsi que les sources utilisées. Les résultats de l'évaluation sont automatiquement enregistrés dans un fichier Excel intitulé Evaluation_CrediBot.xlsx

4. Limites et pistes d'amélioration

4.1 Limites identifiées

- **Base documentaire limitée** : Le chatbot ne peut répondre qu'aux informations présentes dans les documents PDF indexés. Si une information est absente ou incomplète, la réponse peut être imprécise.
- **Mémoire temporaire** : La mémoire est basée sur InMemorySaver, ce qui signifie que l'historique est perdu lorsque l'application est arrêtée.
- **Modèle local limité** : L'utilisation de **Llama 3.2:3B** permet une exécution locale, mais les capacités de raisonnement restent inférieures à celles de modèles plus puissants (GPT-4, Claude, etc.).
- **Outils limités** : L'agent dispose uniquement des outils développés (RAG, recherche Web, calcul). Il ne peut pas accéder directement aux systèmes bancaires, aux bases de données clients ou aux simulateurs de crédit réels.
- **Calculs simplifiés** : Les calculs financiers réalisés par l'agent restent des estimations et ne prennent pas en compte l'ensemble des paramètres utilisés par les établissements bancaires (assurance, frais de dossier, fiscalité, pénalités, etc.).
- **Dépendance à la qualité des documents** : Les performances du RAG dépendent directement de la qualité, de l'actualité et de l'exhaustivité des documents utilisés pour construire la base vectorielle.
-

4.2 Pistes d'amélioration

- Enrichir la base documentaire avec davantage de documents sur le domaine du crédit.
- Utiliser une base vectorielle et une mémoire persistantes pour conserver les données entre les sessions.
- Ajouter de nouveaux outils, tels qu'un simulateur de crédit et un calculateur de capacité d'emprunt.
- Développer une interface utilisateur afin de rendre le chatbot plus simple et plus convivial à utiliser.

Conclusion :

En conclusion, CrediBot démontre l'intérêt de combiner LangGraph, le RAG, les outils et la mémoire pour développer un agent conversationnel intelligent spécialisé dans le domaine du crédit. Ce projet a permis de mettre en pratique les principaux concepts des agents IA et constitue une base solide pour le développement d'assistants intelligents plus avancés